

神经网络

机器学习（双语）

齐飞 (Fei **Qi**) xfred.qi@ieee.org y

李阳 (Yang **Li**)

2024年3月

人工智能学院





神经网络的动机前馈网络函数深

度网络

误差函数 梯度下降



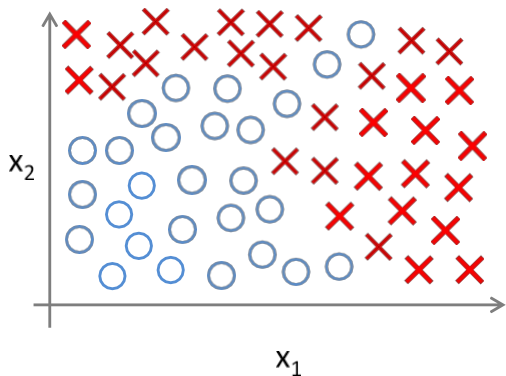
误差反向传播神经网络中的正则

化反向传播的泛化

神经网络的动机



非线性分类与非线性假设



x_1 尺寸
 x_2 #卧室数
 x_3 #楼层
 .
 .
 x100

$$\begin{array}{ccccccc}
 g w_0 & w_1 x_1 & w_2 x_2 & w_3 x_1 x_2 & w_4 x_1^2 & w_5 x_2^2 & w_6 x_1 x_2^2 \\
 & & & \text{宽} & & &
 \end{array}$$



你看到这个：



你看到的是：



但相机看到的是：

194	210	201	212	199	213	215	195	178	158	182	209
180	189	190	221	209	205	191	167	147	115	129	163
114	126	140	188	176	165	152	140	170	106	78	88
87	103	115	154	143	142	149	153	173	101	57	57
102	112	106	131	122	138	152	147	128	84	58	66
94	95	79	104	105	124	129	113	107	87	69	67
68	71	69	98	89	92	98	95	89	88	76	67
41	56	68	99	63	45	60	82	58	76	75	65
20	43	69	75	56	41	51	73	55	70	63	44
50	50	57	69	75	75	73	74	53	68	59	37
72	59	53	66	84	92	84	74	57	72	63	42
67	61	58	65	75	78	76	73	59	75	69	50



汽车

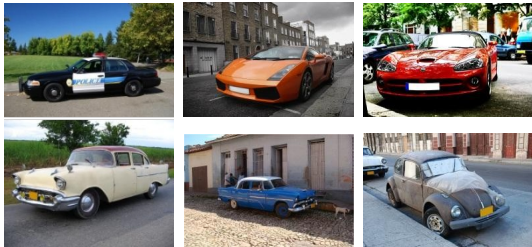


非汽车

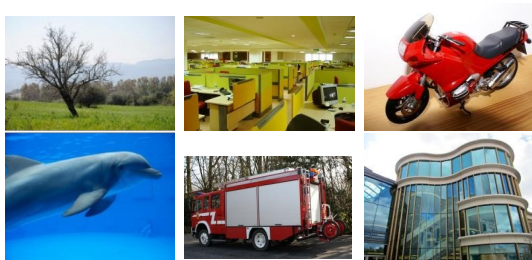




汽车



非汽车

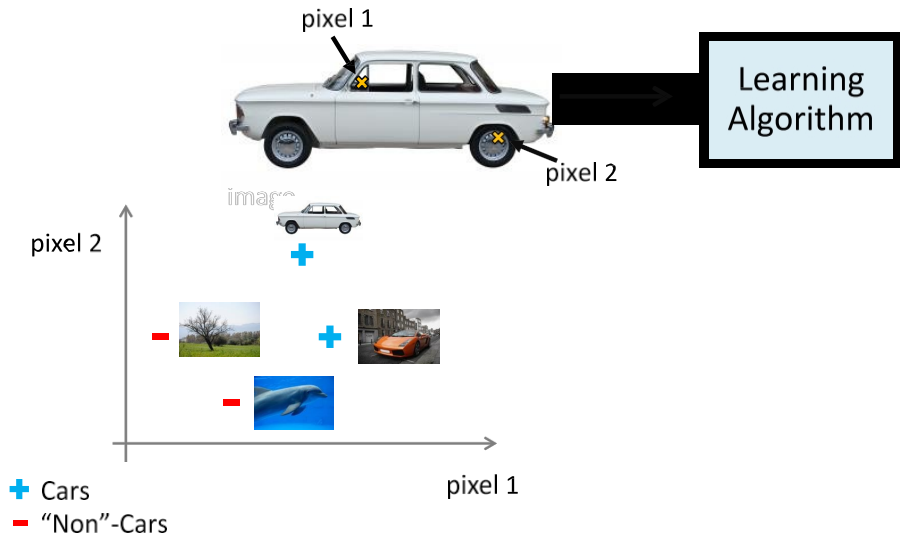


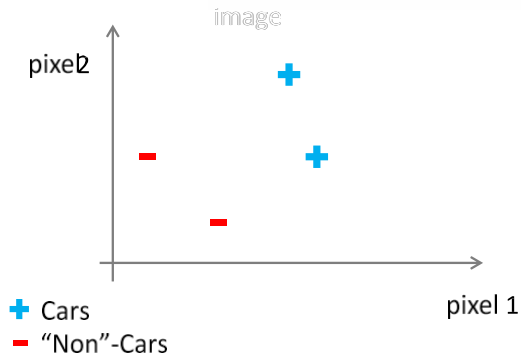
测试：

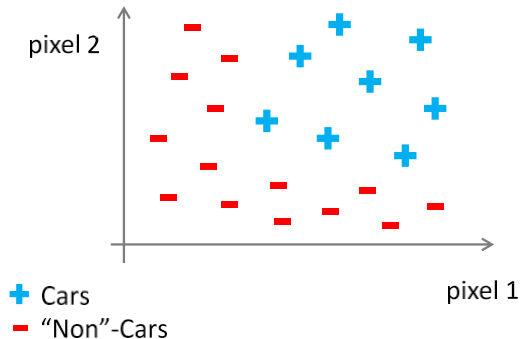


这是什么？



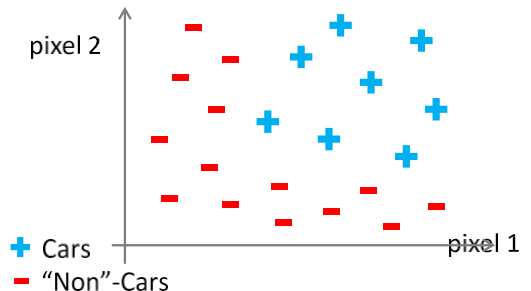








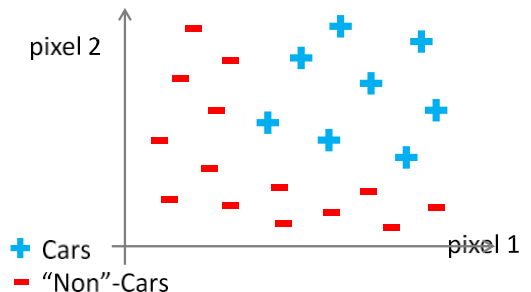
50 50像素图像 2500像素
n 2500 (RGB模式下为7500)



像素 1 强度
像素2强度
x
·
像素2500强度



50 50像素图像 2500像素
n 2500 (RGB时为7500)



像素 1 强度
像素2强度
 x
像素2500强度二次特征(x_i
 x_j)



300万特征

+ Cars
- "Non"-Cars

pixel 1



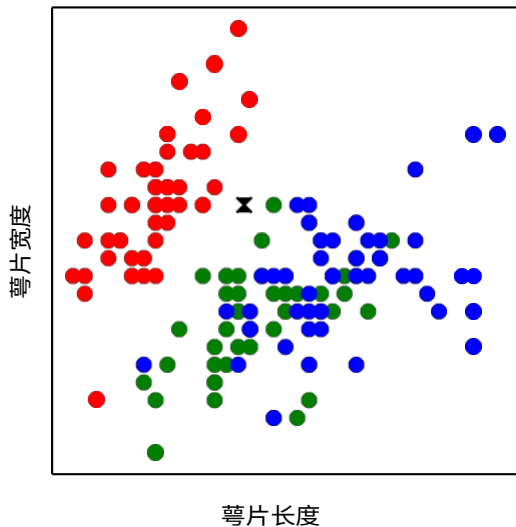
维度诅咒

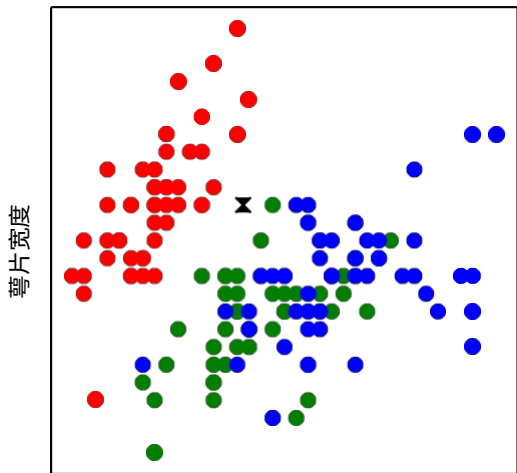
$$y(x, w) = w_0 + w_1 x + w_2 x^2 + \dots + w_M x^M$$

若我们有 D 个输入变量 $x_1, \dots, x_D$ ，则系数最高为三阶的一般多项式将具有如下形式：

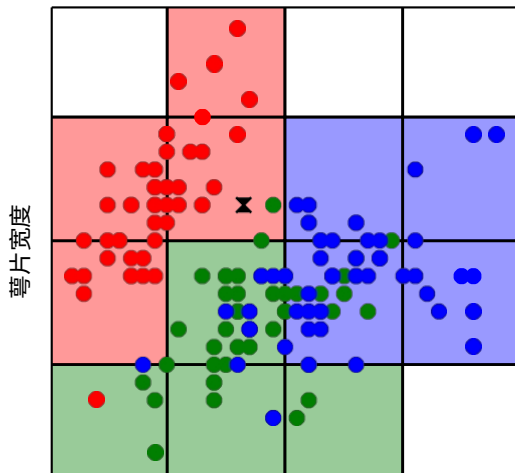
$$y(x, w) = w_0 + \sum_{i=1}^D w_i x_i + \sum_{i=1}^D \sum_{j=1}^D w_{ij} x_i x_j + \sum_{i=1}^D \sum_{j=1}^D \sum_{k=1}^D w_{ijk} x_i x_j x_k$$

系数数量的增长为 $O(D^3)$ 。



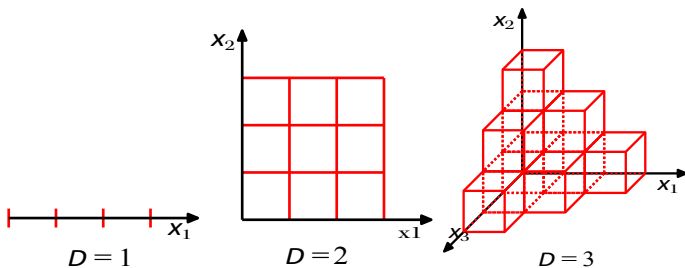
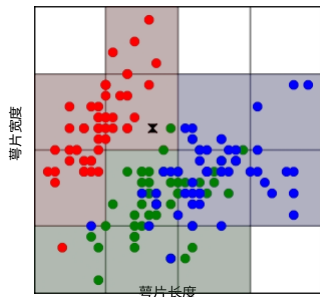


萼片长度



萼片长度

- 将空间区域划分为规则单元，
- #单元数量随空间维度呈指数级增长
- 需要指数级庞大的训练数据量以确保单元格不为空



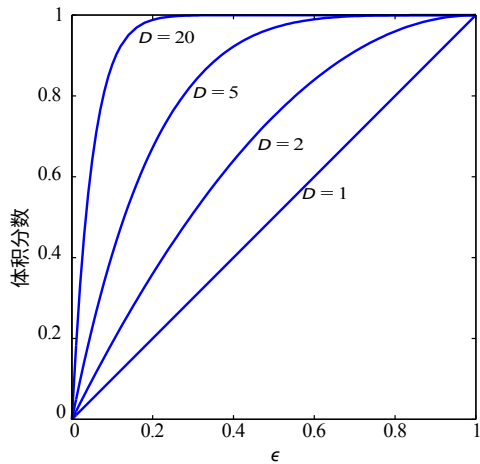


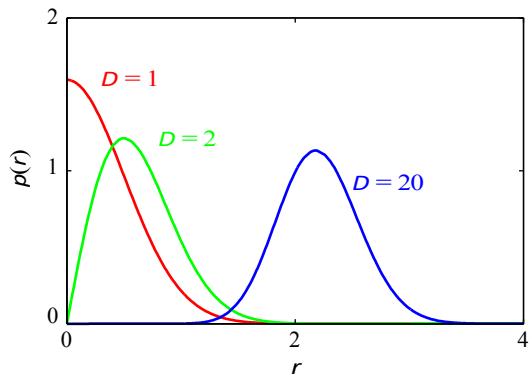
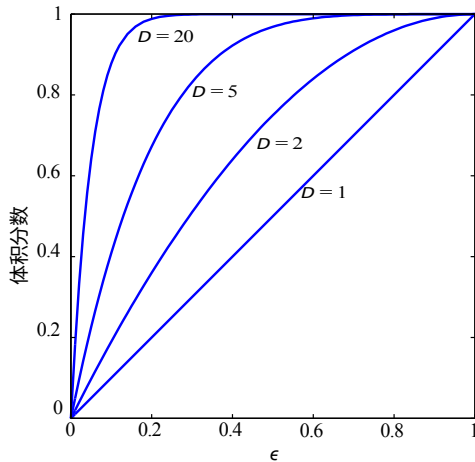
- 考虑半径为 r 的超球体 B_r 的超球体，其所在空间维数为 D 。
- 求半径 $r - \epsilon$ 与 $r + \epsilon$ 之间的超球体体积所占比例。
- 超球体体积 $V_D(r)$ (半径 r , 维数 D) 必须与 r^D 成比例:

$$V_D(r) = K r^D$$

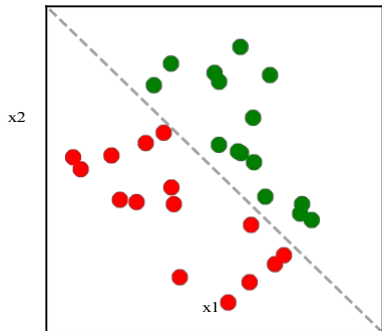
- 所需比例由下列公式给出:

$$\frac{V_D(r + \epsilon) - V_D(r - \epsilon)}{V_D(r)} = \frac{(r + \epsilon)^D - (r - \epsilon)^D}{r^D} \approx \frac{D \epsilon}{r}$$



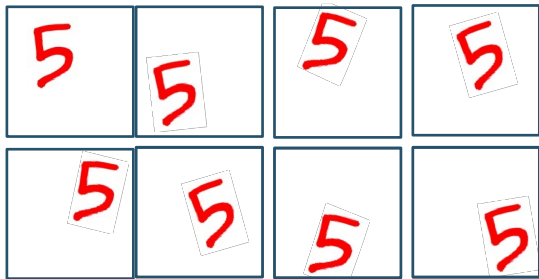


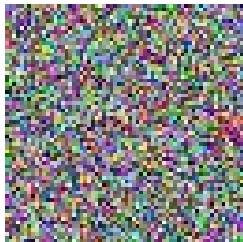
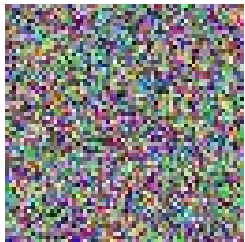
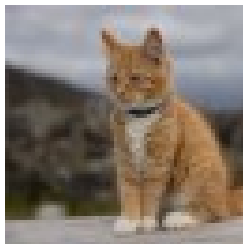
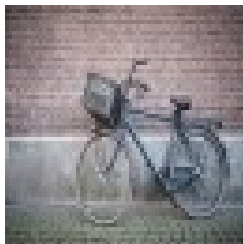
高斯分布 $p(r) \propto r^{D-1} e^{-r^2/2}$



二维数据集示意图 $p_{x_1, x_2} q$



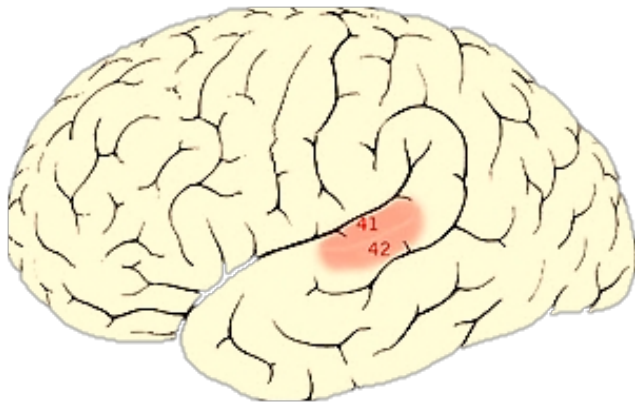






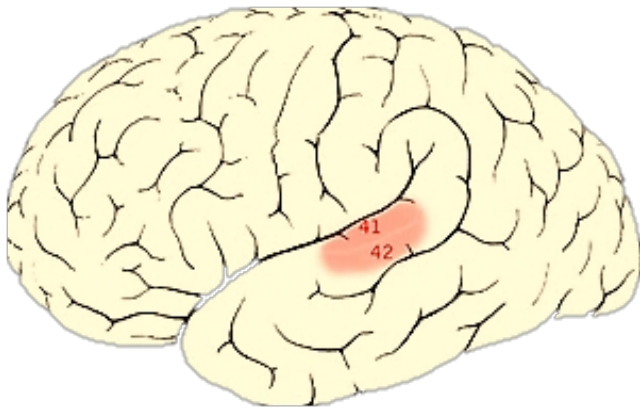
神经网络

- 起源：试图模仿大脑运作的算法。
- 在80年代和90年代初被广泛应用；
- 90年代末期逐渐式微。
- 近年复兴：成为众多应用领域的尖端技术



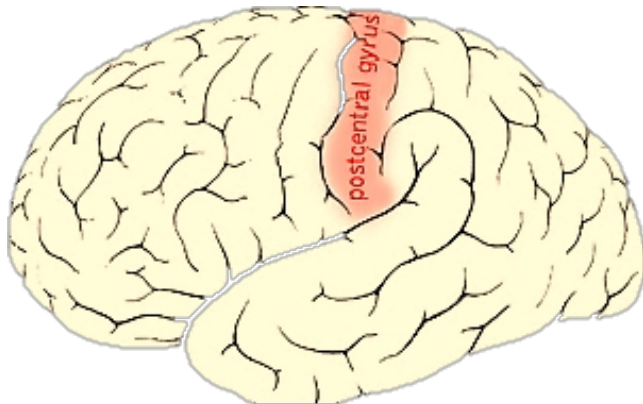
听觉皮层学会看见

- [Roe et al., 1992]



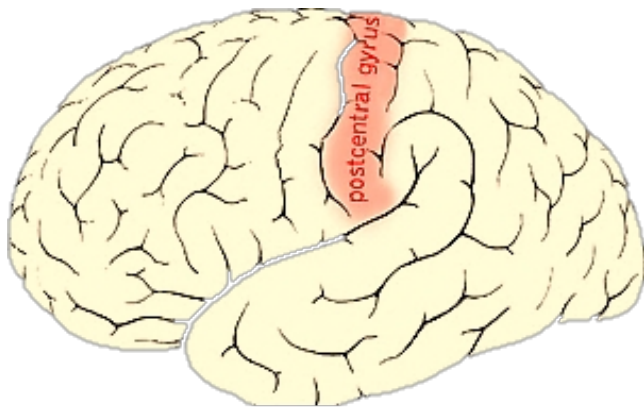
听觉皮层学会看见

- [Roe et al., 1992]



躯体感觉皮层学会看见

- [Metin & Frost, 1989]



躯体感觉皮层学会视觉感知

- [Metin & Frost, 1989]



用舌头看见





用舌头看世界

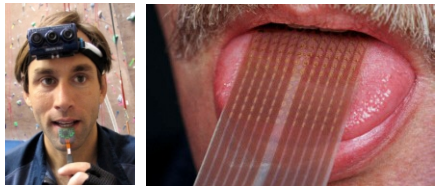


人类回声定位（声纳）





用舌头看世界



人类回声定位（声纳）



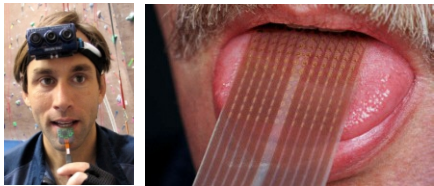
触觉腰带：方向感知



¹ [BrainPort; Welsh & Blasch, 1997; Nagel 等, 2005; Constantine-Paton & Law, 2009]



用舌头看世界



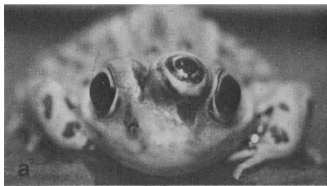
人类回声定位（声纳）



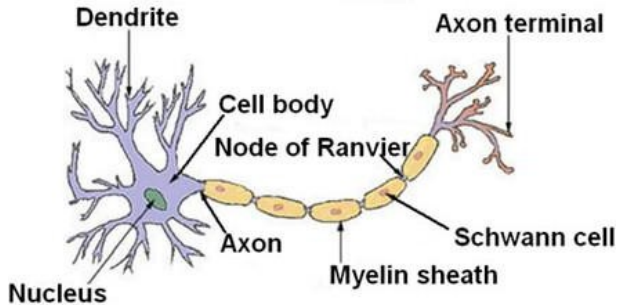
触觉腰带：方向感知



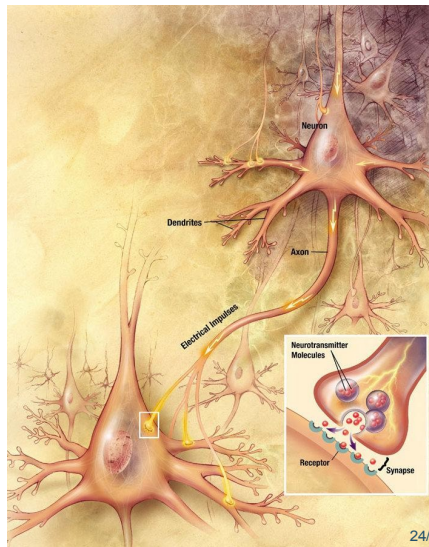
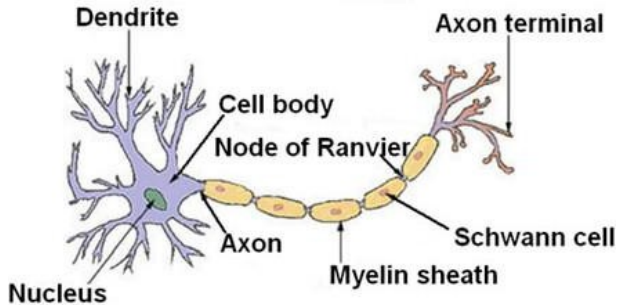
植入第三只眼



¹ [BrainPort; Welsh & Blasch, 1997; Nagel 等, 2005; Constantine-Paton & Law, 2009]



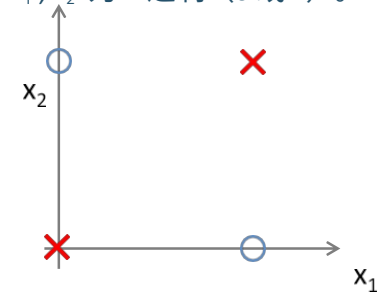
大脑中的神经





直觉：非线性分类（异或/同或）

x_1, x_2 为二进制（0 或 1）。



y x_1 异或 x_2

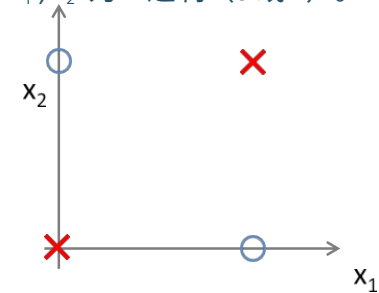
x_1 XNOR x_2

非 $p x_1$ 异或 x_2



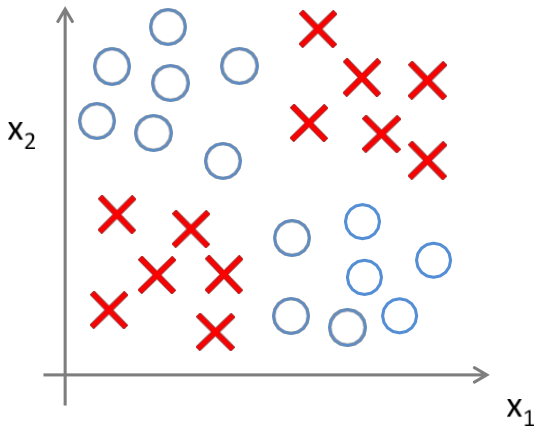
直觉：非线性分类（异或/同或）

x_1, x_2 为二进制（0 或 1）。

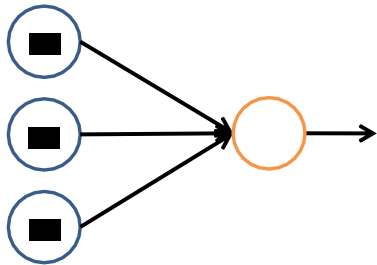


y x_1 异或 x_2
 x_1 XNOR x_2

非 $p x_1$ 异或 x_2 0

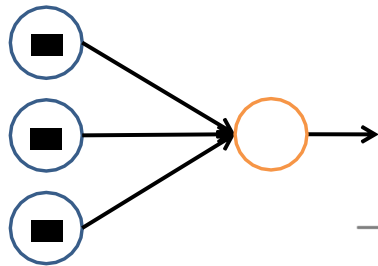


简单示例：AND

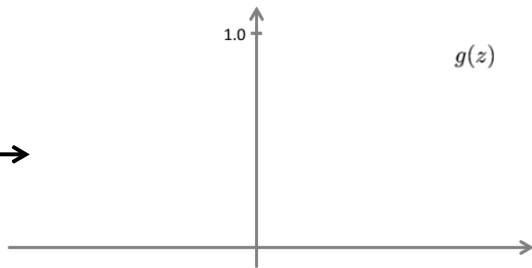


$x_1, x_2 \vdash P_{t0},$
 $1u_{y x_1} \text{ AND}$
 x_2

简单示例: **AND**

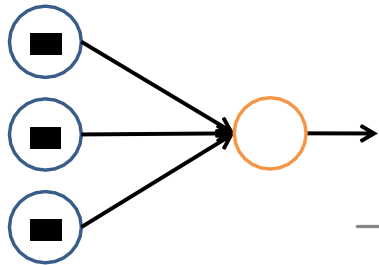


x_1, x_2 $P_{t0}, 1u$
 $y \ x_1 \text{AND} x_2$

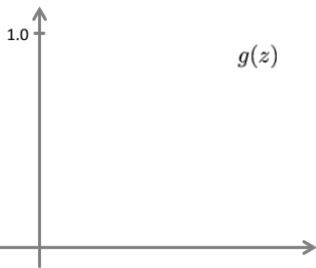




简单示例: AND

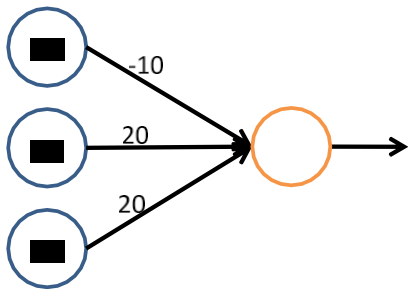


x_1, x_2
 $y = x_1 \text{ AND } x_2$

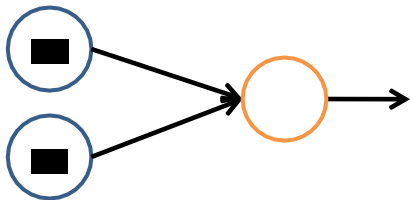


x_1	x_2	h_w
0	0	0
0	1	0
1	0	0
1	1	1

示例：OR 函数

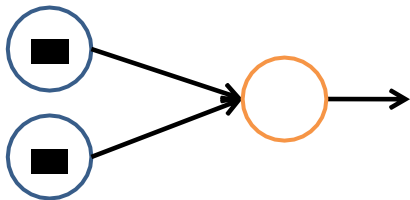


x_1	x_2	h_w	pxq
0	0		
0	1		
1	0		
1	1		



$$h_w \quad p x q \quad g p 10 \quad 20 x_1 \quad q$$

$$\begin{array}{c} \hline x_1 \quad h_w \quad p x q \\ 0 \\ 1 \\ \hline \end{array}$$



$$h_w p x q \quad g p 10 \ 20 x_1 q$$

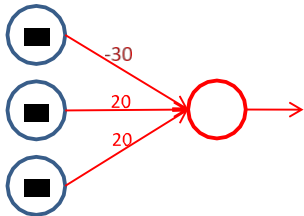
$$\begin{array}{c} \hline x_1 \quad h_w \quad p x q \\ 0 \\ 1 \\ \hline \end{array}$$

$$p \text{NOT } x_1 \ q \ \text{AND} \ p \text{NOT } x_2 \ q$$

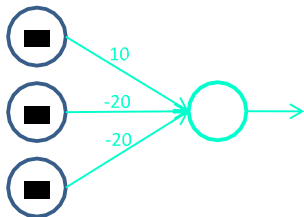
组合运算: x_1 XNOR x_2



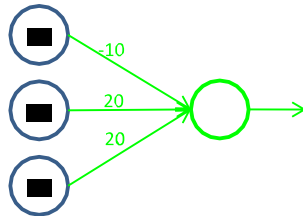
x_1 AND x_2



$p \neg x_1$ q 且 $p \neg x_2$ q



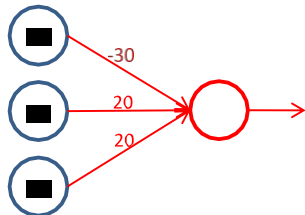
x_1 OR x_2





组合运算: x_1 XNOR x_2

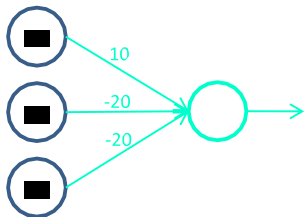
x_1 AND x_2



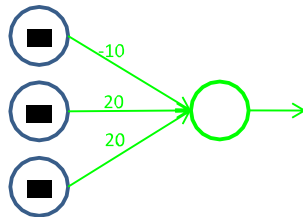
x_1 XNOR x_2



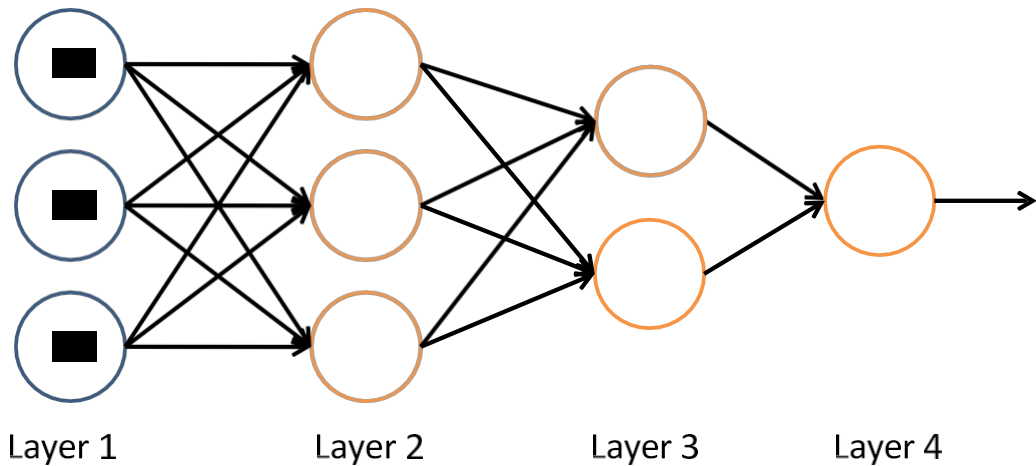
$p \text{ 非 } x_1 \text{ 且 } p \text{ 非 } x_2 \text{ 且 } q$



x_1 或 x_2



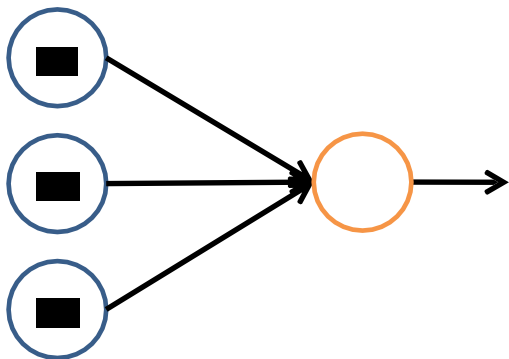
x_1	x_2	a_1^{p2q}	a_2^{p2q}	h_w
pxq				
0	0			
0	1			
1	0			
1	1			



前馈网络功能



神经元模型：逻辑单元



$\mathbf{x}$	x_0	$\mathbf{w}$	w_0
	x_1		w_1
	x_2		w_2
	x_3		w_3

S形（逻辑）激活函数



- 亦称多层感知器
- “神经网络”一词源于人们试图寻找生物系统中信息处理的数学表示形式
- 该术语已被广泛应用于涵盖多种不同模型



- 回归与分类的线性模型基于固定非线性基函数 $\varphi_j(\mathbf{x})$ 的线性组合，其形式为

$$y(\mathbf{x}, \mathbf{w}) = f \left(\sum_{j=1}^M w_j \varphi_j(\mathbf{x}) \right)$$

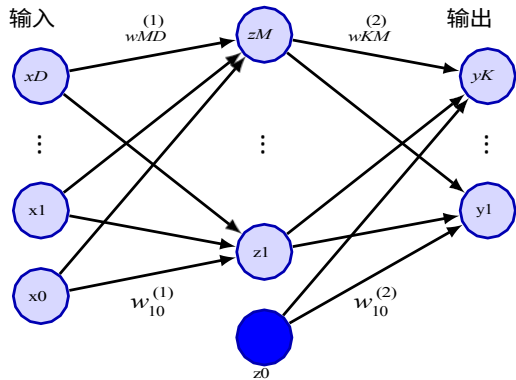
- 其中 $f(\cdot)$ 在分类情况下为非线性激活函数
- 而在回归问题中则是恒等函数。



- 神经网络的核心思想在于选择基函数 $\varphi_j x$ ，使得
这些基函数本身具有可学习参数，并在训练过程中允许这些参数与系数 tw_j U 共同调整。
- 神经网络采用遵循相同形式的基函数
- 使得每个基函数本身都是输入线性组合的非线性函数，
- 其中线性组合中的系数为自适应参数。



具有两层可学习参数的神经网络 隐藏单元

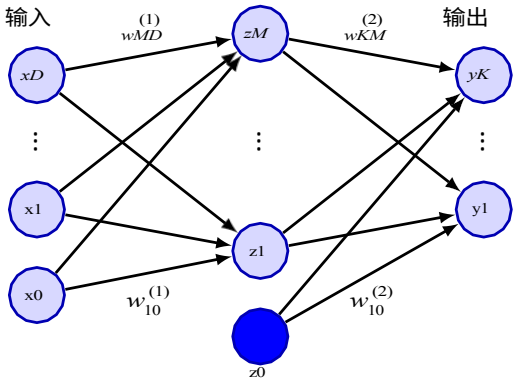


$$a_j = \sum_{i=1}^D w_{ji} x_i + w_{j0}$$



前馈网络功能

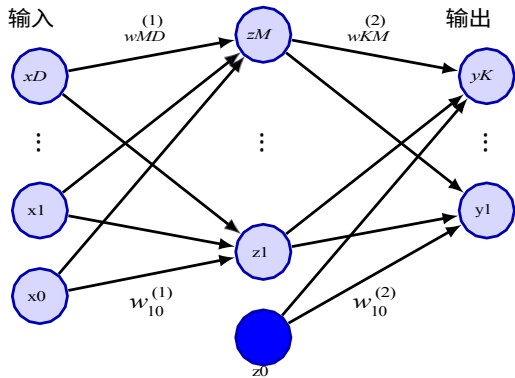
具有两层可学习参数的神经网络
隐藏单元



$$a_j = \sum_{i=1}^D w_{ji}^{p1q} x_i$$
$$z_j^{p1q} = \max\{0, a_j^{p1q}\}$$
$$h_{p1q} = \{z_j^{p1q}\}_j$$
$$y_q = \sum_{j=0}^K w_{j0}^{p1q} z_j^{p1q}$$



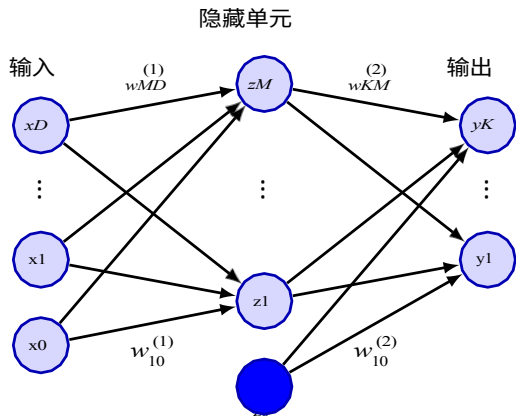
具有两层可学习参数的神经网络
隐藏单元



$$\begin{aligned}
 & \text{D} \\
 & a_j \quad w_{ji}^{p1q} \quad x_i \quad w_{j0}^{p1q} \\
 & i1 \\
 & z_j^{p1q} \quad h p a_j^{p1q} q. \\
 & p2q \quad M \quad p2q \quad p2q \\
 & ak \quad w_{kj}^{p2q} \quad z_j \quad w_{k0}^{p2q} \\
 & j1
 \end{aligned}$$



参数矩阵

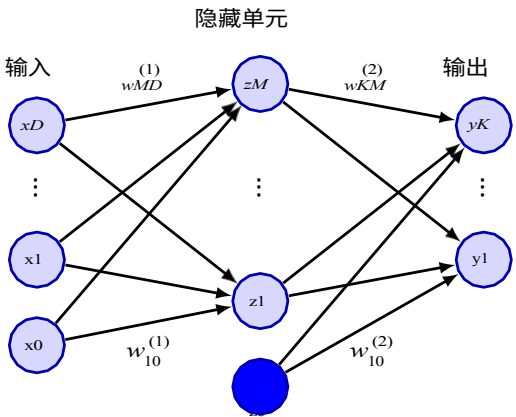


$$a_j = \sum_{i=0}^D w_{ji} x_i$$



前馈网络功能

参数矩阵



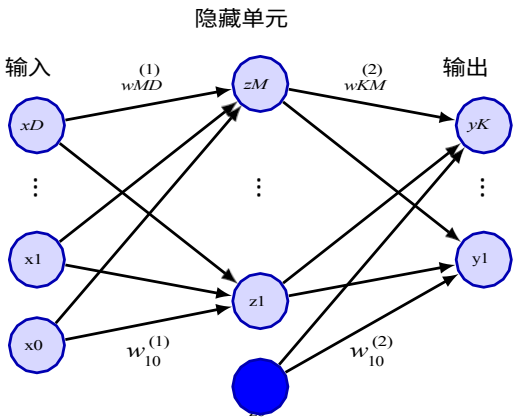
$$y_k = \sigma \left(\sum_{j=0}^M w_{kj}^{(2)} h_j \right)$$

$$h_j = \sigma \left(\sum_{i=0}^D w_{ji}^{(1)} x_i \right)$$



前馈网络功能

参数矩阵



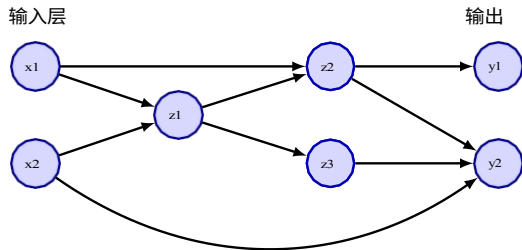
$$y_k = \sigma \left(\sum_{j=0}^M w_{kj}^{(2)} z_j \right) = \sigma \left(\sum_{j=0}^M w_{kj}^{(2)} \left(\sum_{i=0}^D w_{ji}^{(1)} x_i \right) \right)$$



- 通过考虑更复杂的网络图，我们可以开发出更通用的网络映射。
- 这些映射必须限制在前馈架构内，
- 即不含闭合有向环的路径，以确保输出为成为输入的*确定性*函数。

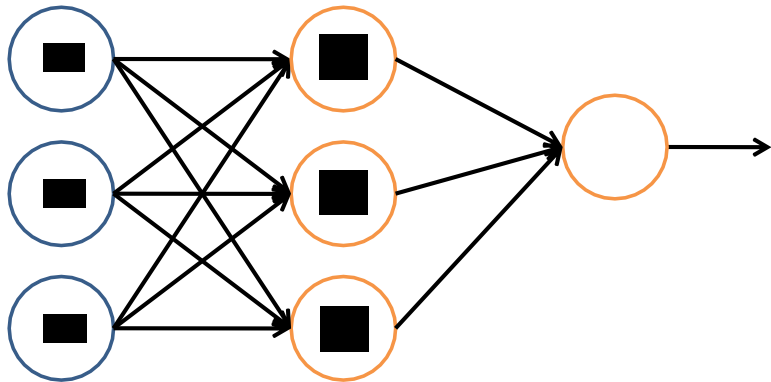


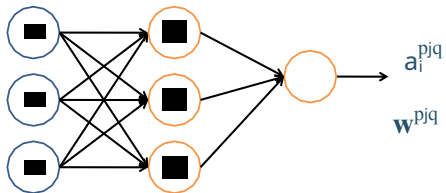
- 具有通用前馈拓扑结构的神经网络示例。



- A_{pkq} 表示节点 k 的祖先集合

$$z_k = h \left(\sum_{j \in A_{pkq}} w_{kj} z_j + b_k \right)$$

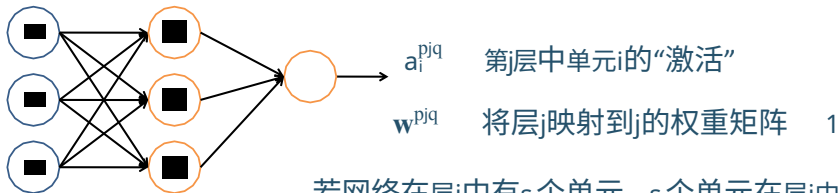




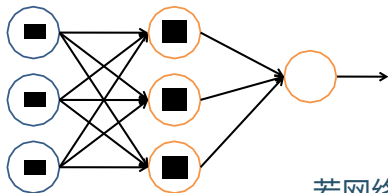
层j中单元i的“激活”

$\mathbf{w}^{pq}$

将层j映射到j的权重矩阵 1



若网络在层j中有 s_j 个单元， s_{j-1} 个单元在层j-1中，则
 w^{pq} 的维度为 $s_{j-1} \times s_j$ 。



a_i^{pq} 层中单元i的“激活”

w^{pq} 权重矩阵，将层j映射到层j+1

若网络在层j中有 s_j 个单元，且层j+1的单元数为 s_{j+1} 个，则

w^{pq} 的维度为 $s_{j+1} \times s_j$ 。

$$\begin{aligned}
 & z_1^{p2q} = h p a_1^{p2q} h = w_{10}^{p1q} x_0 + w_{11}^{p1q} x_1 + w_{12}^{p1q} x_2 + w_{13}^{p1q} x_3 \\
 & z_2^{p2q} = h p a_2^{p2q} h = w_{20}^{p1q} x_0 + w_{21}^{p1q} x_1 + w_{22}^{p1q} x_2 + w_{23}^{p1q} x_3 \\
 & z_3^{p2q} = h p a_3^{p2q} h = w_{30}^{p1q} x_0 + w_{31}^{p1q} x_1 + w_{32}^{p1q} x_2 + w_{33}^{p1q} x_3 \\
 & y_w^{pxq} = z_1^{p3q} = a_{p3q}^{p3q} h = w_{p30}^{p2q} z_0 + w_{p31}^{p2q} z_1 + w_{p32}^{p2q} z_2 + w_{p33}^{p2q} z_3
 \end{aligned}$$

$$\begin{array}{cc} p^2q & p^2q \\ p^2q & p^2q \\ p^2q & \end{array}$$

Z

Z

W

1

2

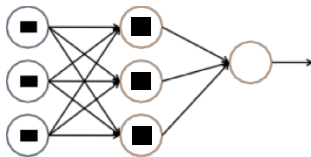
5

13 z3

$p2q$



前向传播：向量化实现



$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \mathbf{a}^{p2q} = \begin{bmatrix} a_1^{p2q} \\ a_2^{p2q} \\ a_3^{p2q} \end{bmatrix}$$

$$z_1^{p2q} = h \begin{bmatrix} w_{10}^{p1q} x_0^{p1q} & w_{11}^{p1q} x_1^{p1q} & w_{12}^{p1q} x_2^{p1q} & w_{13}^{p1q} x_3^{p1q} \end{bmatrix}$$

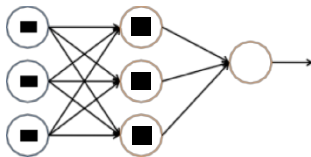
$$z_2^{p2q} = h \begin{bmatrix} w_{20}^{p1q} x_0^{p1q} & w_{21}^{p1q} x_1^{p1q} & w_{22}^{p1q} x_2^{p1q} & w_{23}^{p1q} x_3^{p1q} \end{bmatrix}$$

$$z_3^{p2q} = h \begin{bmatrix} w_{30}^{p1q} x_0^{p1q} & w_{31}^{p1q} x_1^{p1q} & w_{32}^{p1q} x_2^{p1q} & w_{33}^{p1q} x_3^{p1q} \end{bmatrix}$$

$$y_w^{pxq} = h \begin{bmatrix} w_{10}^{p2q} z_0^{p2q} & w_{11}^{p2q} z_1^{p2q} & w_{12}^{p2q} z_2^{p2q} & w_{13}^{p2q} z_3^{p2q} \end{bmatrix}$$



前向传播：向量化实现



$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \mathbf{a}^{p2q} = \begin{bmatrix} a_1^{p2q} \\ a_2^{p2q} \\ a_3^{p2q} \end{bmatrix}$$

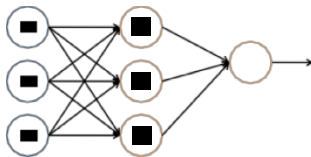
$$\begin{aligned} z_1^{p2q} &= h^{p1q} w_{10}^{p1q} x_0 + w_{11}^{p1q} x_1 + w_{12}^{p1q} x_2 + w_{13}^{p1q} x_3 \\ z_2^{p2q} &= h^{p1q} w_{20}^{p1q} x_0 + w_{21}^{p1q} x_1 + w_{22}^{p1q} x_2 + w_{23}^{p1q} x_3 \\ z_3^{p2q} &= h^{p1q} w_{30}^{p1q} x_0 + w_{31}^{p1q} x_1 + w_{32}^{p1q} x_2 + w_{33}^{p1q} x_3 \end{aligned}$$

$$\mathbf{a}^{p2q} = \mathbf{w}^{p1q} \mathbf{x}$$

$$\mathbf{y}_w = \mathbf{p} \mathbf{x} \mathbf{q} \quad h \mathbf{w}_{10}^{p2q} z_0^{p2q} \quad \mathbf{w}_{11}^{p2q} z_1^{p2q} \quad \mathbf{w}_{12}^{p2q} z_2^{p2q} \quad \mathbf{w}_{13}^{p2q} z_3^{p2q}$$



前向传播：向量化实现



$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \mathbf{a}^{p2q} = \begin{bmatrix} a_1^{p2q} \\ a_2^{p2q} \\ a_3^{p2q} \end{bmatrix}$$

$$\begin{aligned} z_1^{p2q} &= h^{p1q} w_{10}^{p1q} x_0 + w_{11}^{p1q} x_1 + w_{12}^{p1q} x_2 + w_{13}^{p1q} x_3 \\ z_2^{p2q} &= h^{p1q} w_{20}^{p1q} x_0 + w_{21}^{p1q} x_1 + w_{22}^{p1q} x_2 + w_{23}^{p1q} x_3 \\ z_3^{p2q} &= h^{p1q} w_{30}^{p1q} x_0 + w_{31}^{p1q} x_1 + w_{32}^{p1q} x_2 + w_{33}^{p1q} x_3 \end{aligned}$$

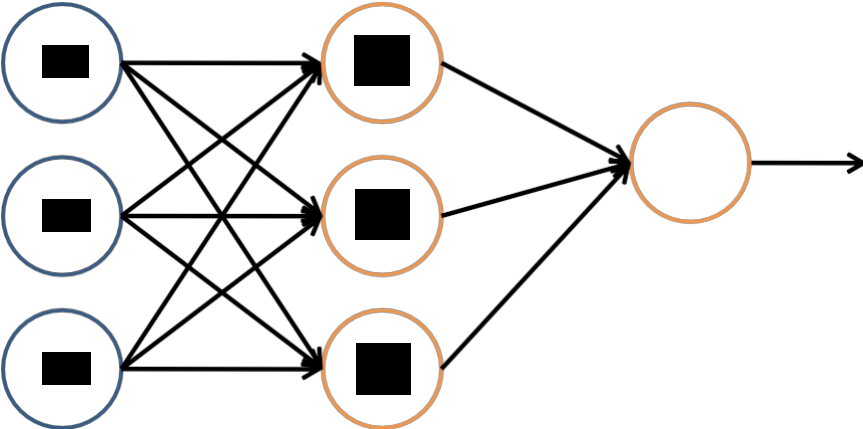
$$y_w^{p2q} = \mathbf{p}^{p2q} \mathbf{x}^{p2q} + h^{p2q} w_{10}^{p2q} z_0^{p2q} + w_{11}^{p2q} z_1^{p2q} + w_{12}^{p2q} z_2^{p2q} + w_{13}^{p2q} z_3^{p2q}$$

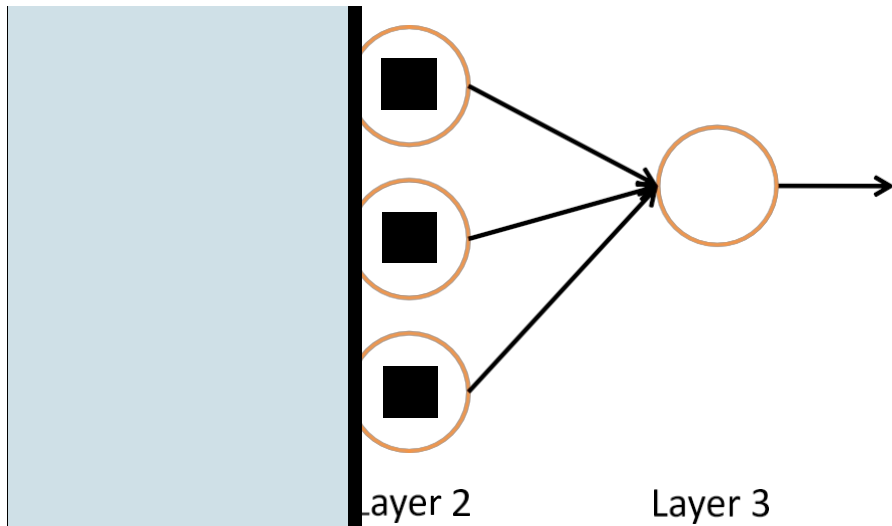
$$\mathbf{a}^{p2q} = \mathbf{w}^{p1q} \mathbf{x}$$

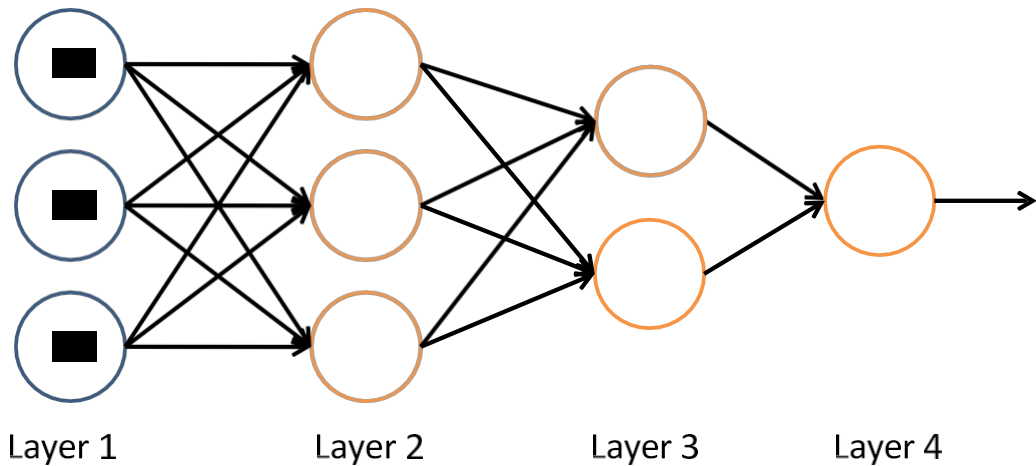
$$z^{p2q} = h^{p2q} \mathbf{a}^{p2q}$$

$$\mathbf{a}^{p3q} = \mathbf{w}^{p2q} \mathbf{z}^{p2q}$$

$$y_w^{p3q} = \mathbf{p}^{p3q} \mathbf{x}^{p3q} + h^{p3q} \mathbf{a}^{p3q} \mathbf{q} \quad 41/184$$







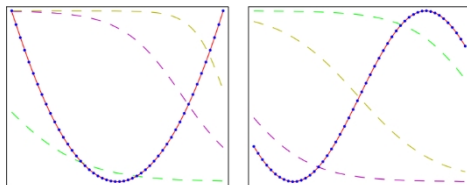


- 前馈网络的逼近特性已被广泛研究。
- 因此，神经网络被称为通用逼近器。
- 例如，一个具有线性输出的两层网络，只要具有足够多的隐藏单元，就可以以任意精度均匀地近似任何紧输入域上的连续函数。
- 该结论适用于多种隐藏单元激活函数，但排除多项式函数。



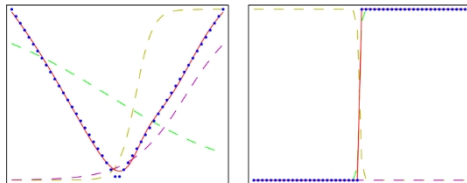
- MLP可近似四种不同函数：

(a) $f(x) = x^2$, (b) $f(x) = \sin(x)$, (c) $f(x) = |x|$, (d) $f(x) = H(x)$.



(a)

(b)



(c)

(d)

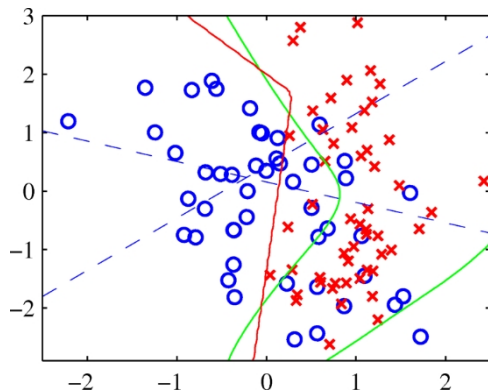
- $N = 50$ 个数据点，以蓝色圆点显示
- 以及相应的 $f(x)$
- 一个具有3个隐藏单元的两层网络，采用tanh
- 红色：预测值，三隐层单元以虚线



曲线表示



- 二元分类：2个输入，2个tanh隐藏层，sigmoid输出层。



蓝色虚线显示每个隐藏层的 $z = 0.5$ 等值线，红色实线表示网络的 $y = 0.5$ 决策面。

作为对比，绿色曲线表示根据生成数据的分布计算出的最优决策边界。



- 输出单元的激活函数取决于所建模的分布类型。



- 输出单元的激活函数取决于所建模分布的类型。
- 对于隐藏单元，唯一的要求是它们必须可微分。



- 输出单元的激活函数取决于所建模的分布类型。
- 对于隐藏单元，唯一的要求是它们必须可微分
- 大多数情况下，所有隐藏单元都采用相同的激活函数



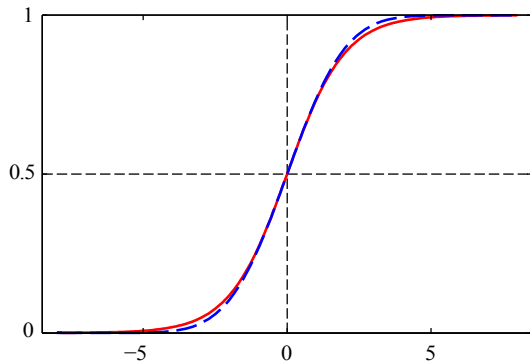
- 输出单元的激活函数由所建模的分布类型决定。
- 对于隐藏单元，唯一的要求是它们需要可微分
- 在多数情况下，所有隐藏单元都采用相同的激活函数
- 最简单的选择是恒等函数，这意味着所有隐藏单元都成为线性单元。



- 输出单元的激活函数取决于所建模的分布类型。
- 对于隐藏单元，唯一的要求是它们必须可微分
- 在大多数情况下，所有隐藏单元都将采用相同的激活函数
- 最简单的选择是恒等函数，这意味着所有隐藏单元都将线性化。
- 对于任何此类网络，我们总能找到一个没有隐藏单元的等效网络。



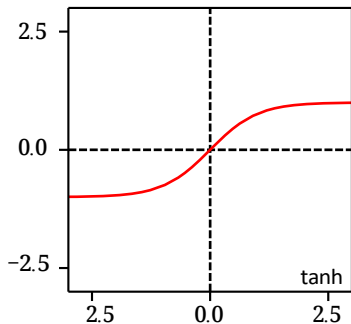
Sigmoid



$$\sigma(p_a q) = \frac{1}{1 + \exp(-p_a q)}$$

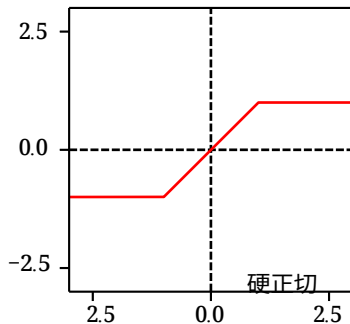


tanh



$$\tanh x = \frac{e^{2x} - 1}{e^{2x} + 1}$$

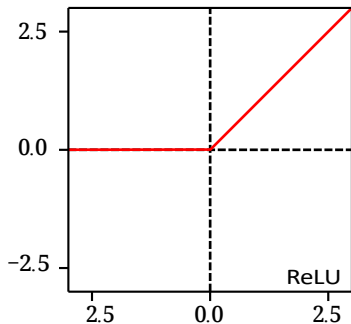
硬正切



$$\text{hard tangent}(x) = \begin{cases} -1 & x < 0 \\ 0 & x = 0 \\ 1 & x > 0 \end{cases}$$

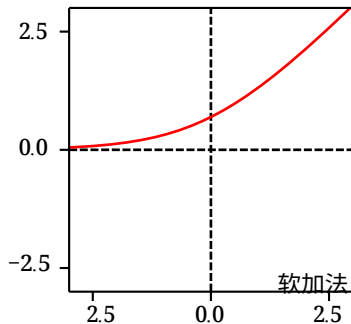


整流线性单元



hpaq maxp0,

软加法/软ReLU



aq

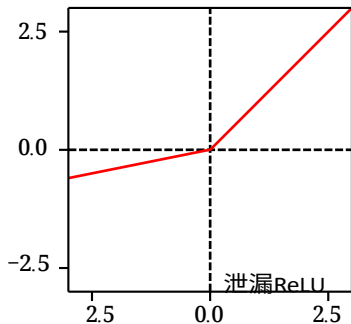


$\text{hpaq}_{\ln 1}$

exppaq

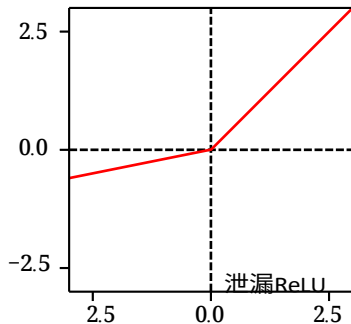


泄漏ReLU



hpaq $\max(p_0, a)$ $\min(p_0, a)$

绝对值



hpaq $|a|$

深度网络



我们可以将两层网络架构扩展到任意有限层数 L

$$y = p_{x,w} q = f_W^{p2q} h W^{p1q} x$$

$$z^{plq} = h^{plq} W^{plq} z^{pl-1q}$$

- h^{plq} 表示与层 l 相关的激活函数，
- W^{plq} 表示对应的权重与偏置参数矩阵。
- $z^{p0q} = x$ 表示输入向量
- $z^{pLq} = y$ 表示输出向量。



分层表示

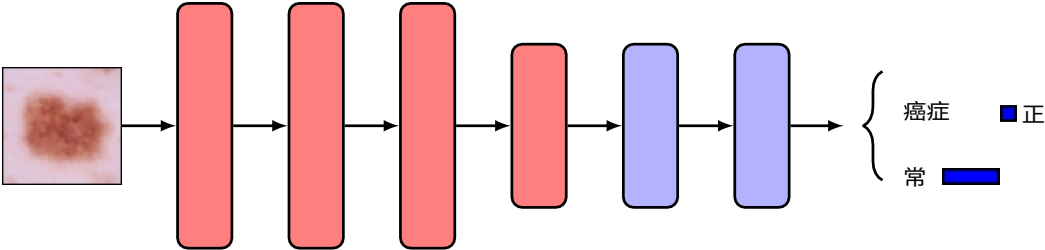
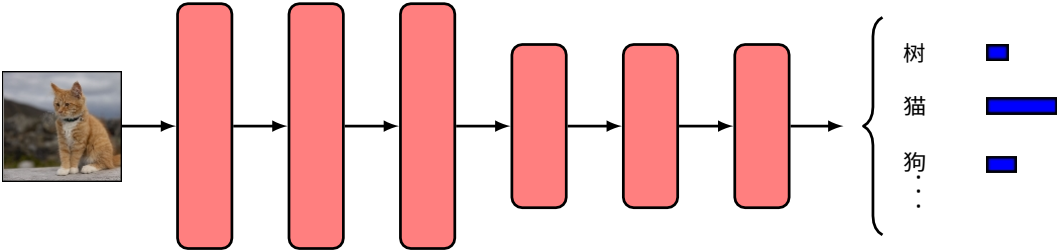
输出通过分层表示与输入空间相关联

分布式表示

网络可能学习到另一种表示方式，其中隐藏单元的组合代表特征，从而使具有M个单元的隐藏层能够表示 2^M 种不同特征，其数量随单元数量呈指数级增长。

表示学习

我们可以将深度神经网络的逐层结构视为对数据进行变换，从而更容易解决所需任务。





信息**NCE**损失（噪声对比估计，**NCE**）

$$E_{p_w, q} \left[-\ln \frac{\exp(f_w(p, q))}{\exp(f_w(p, q)) + \sum_{n=1}^N \exp(f_w(p, q_n))} \right]$$

实验函数 f_w

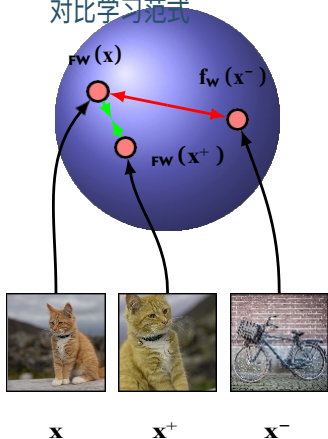
- 给定数据点 x 称为锚点
- 另一个数据点 x' 与 x 共同构成正配对
- 一组数据点 $x_1, \dots, x_N$ ，其中每个点构成负配对

与 x 相关联

- 余弦相似度 $\exp(f_w(p, q))$



对比学习范式



$$E_{p(w, q)} \ln \frac{\exp(f_w^T p x q^T f_w p x) q u}{\exp(f_w^T p x q^T f_w p x) q u} \prod_{n=1}^N \exp(f_w^T p x q^T f_w p x_n) q u$$

误差函数



误差函数：回归

- 让我们从回归开始
- 考虑单一目标变量 t ，其取值范围为任意实数

$$p(t|x, \mathbf{w}, \mathbf{q}) \sim \mathcal{N}(t | \mathbf{y}^T \mathbf{p}(x, \mathbf{w}, \mathbf{q}), \sigma^2)$$

- 最大化似然函数等价于最小化由下列公式给出的平方和误差函数

$$E_p(\mathbf{w}, \mathbf{q}) = \frac{1}{2} \sum_{n=1}^N (t_n - \mathbf{y}^T \mathbf{p}(x_n, \mathbf{w}, \mathbf{q}))^2$$

- 若存在多个目标变量，且假设它们相互独立

$$E_p(\mathbf{w}, \mathbf{q}) = \frac{1}{2} \sum_{n=1}^N \sum_{k=1}^K (t_{n,k} - \mathbf{y}^T \mathbf{p}(x_n, \mathbf{w}, \mathbf{q}))^2$$



- 误差函数（由负对数似然函数给出）与输出单元激活函数之间存在自然对应关系。
- 在回归问题中，网络的输出激活函数为恒等函数，因此 $y_{(k)} = a_k$ 。
- 对应的平方和误差函数具有如下性质：

$$E = \frac{1}{2} \sum_k (y_k - t_k)^2$$



误差函数：二元分类

- 考虑二元分类的情况
- 此时目标在输入条件下的条件分布为伯努利分布，形式如下：

$$p(t|x, \mathbf{w}, \mathbf{q}) = y_{\mathbf{p}\mathbf{x}} \mathbf{w}\mathbf{q}^{t-1} + (1 - y_{\mathbf{p}\mathbf{x}}) \mathbf{w}\mathbf{q}^{1-t}.$$

- 若训练集由独立观测构成，
- 则误差函数为交叉熵误差

$$E_{\mathbf{p}\mathbf{w}\mathbf{q}} = - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln (1 - y_n)]$$

其中 y_n 表示 $y_{\mathbf{p}\mathbf{x}_n}$ ， $\mathbf{w}\mathbf{q}$ 。

多类分类



行人



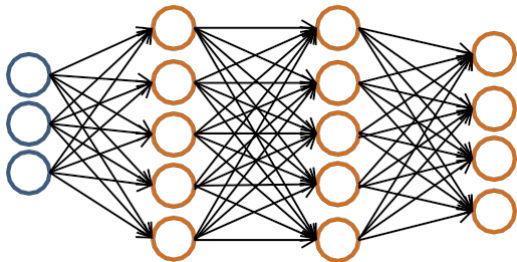
汽车



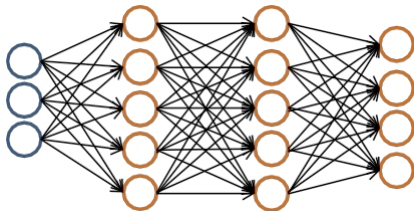
摩托车



行李箱



多类分类: 1-of-K



$$y_{px_n}, wq \in \mathbb{R}^4$$

- 求 y_{px_n}, wq

1	0	0
0	1	0
0	0	1

, y_{px_n}, wq

0	0	0
1	0	0
0	1	0

, 等等

当行人时, 当汽车时, 当摩托车时



- 训练集:

$\mathbf{x} \quad \mathbf{t} \quad \mathbf{x} \quad \mathbf{t} \quad \mathbf{x} \quad \mathbf{t}_N$
 $1, 1, 2, 2, \dots, N,$

- $\mathbf{t}_n$ oneof

1	0	0	0
0,	1,	0,	0
0	0	1	0
0	0	0	1



误差函数：多类分类

- 如果需要执行K个独立的二元分类任务，
- 则可采用具有K个输出的网络，每个输出采用逻辑sigmoid激活函数。
- 每个输出均为二元类别标签 $t_k \in \{0, 1\}$ 。
- 若假设类标签相互独立，则对应似然函数的负对数可得如下误差函数：

$$E_{\text{pwq}} = -\sum_{n=1}^N \sum_{k=1}^K t_{nk} \ln y_{nk} - \sum_{n=1}^N \sum_{k=1}^K (1-t_{nk}) \ln (1-y_{nk})$$

其中 y_{nk} 表示 y_k 在 x_n 处的输出， w_{qk} 表示第 q 个输出单元对第 k 个类别的权重。



误差函数：多类分类

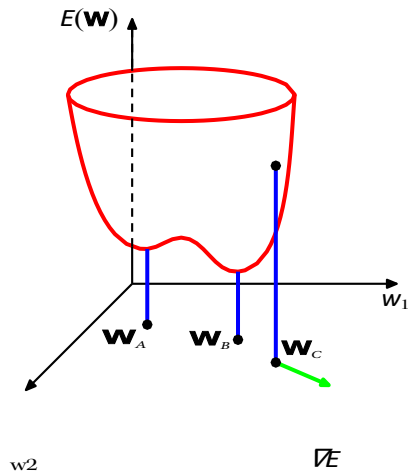
- 考虑标准的多类分类问题，其中每个输入被分配到 K 个互斥类中的一个

$$t_{kn} = \ln y_k \mathbf{p}_{\mathbf{x}_n}, \mathbf{w}_k.$$

- 我们看到，输出单元的激活函数（对应于规范链接）由softmax函数给出：

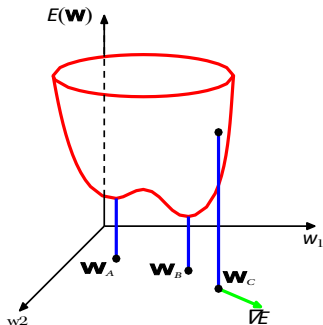
$$y_k \mathbf{p}_{\mathbf{x}, \mathbf{w}} = \frac{\exp a_k \mathbf{p}_{\mathbf{x}, \mathbf{w}}}{\sum_j \exp a_j \mathbf{p}_{\mathbf{x}, \mathbf{w}}}$$

梯度下降



- 误差函数 $E(w)$ 的几何视图
作为覆盖权重空间的表面。
- 点 w_A 是局部极小值
- 而点 w_B 是全局极小值。
- 在任意点 w_C 处，误差曲面的局部梯度由向量 ∇E 给出。

pq



- 在整个 w 空间中，对应于误差函数最小值的极小值称为全局极小值。
- 任何其他对应于更高误差函数值的极小值则称为局部极小值。
- 深度神经网络的误差表面可能极其复杂，
- 人们曾认为基于梯度的方法可能陷入次优局部极小值。



误差曲面：局部二次逼近

- 考虑权重空间中某点 $\hat{w}$ 附近的 $E(w)$ 泰勒展开式

$$E(w) \approx E(\hat{w}) + \nabla E(\hat{w})^T (w - \hat{w}) + \frac{1}{2} (w - \hat{w})^T H(\hat{w}) (w - \hat{w})$$

- 其中已省略三次及更高次项。
- 此处定义梯度向量 b 为梯度 E 在梯度向量 $\hat{w}$ 处的值: $b = \nabla E|_{\hat{w}}$
- 海森矩阵定义为二阶导数的对应矩阵

$$H(\hat{w}) = \nabla \nabla E(w)|_{\hat{w}}$$

- 梯度的对应局部逼近为

$$\nabla E(w) \approx b + H(\hat{w})(w - \hat{w})$$



误差曲面：局部二次逼近

- 考虑误差函数极小值点 w 附近的局部二次逼近特例。
- 在此情况下不存在线性项，因为在点 w 处有 $\nabla E = 0$ 。

$$E(p; w, q) \approx E(p; w) + \frac{1}{2} (w - w^T)^T H (w - w)$$



误差曲面：局部二次逼近

- 要从几何角度解释这一点，请考虑海森矩阵的特征值方程

$$Hu_i = \lambda_i u_i$$

- 其中特征向量 u_i 构成完备正交规范基，满足 $u_i^T u_j = \delta_{ij}$ 。

- 现在将 $w = w$ 展开为特征向量的线性组合

$$w = \sum_i \alpha_i u_i .$$



误差曲面：局部二次逼近

- 这可视作坐标系的变换
- 其中原点平移至点 w ,
- 坐标轴旋转至与特征向量对齐。
- 误差函数可表示为如下形式

$$E_{pw}(\mathbf{q}) = E_{pw} + \frac{1}{2} \mathbf{q}^T \mathbf{H} \mathbf{q} - i\lambda \mathbf{q}^2.$$



误差曲面：局部二次逼近

- 若且仅若满足下列条件，则称矩阵H为正定矩阵：

$$\mathbf{v}^T \mathbf{H} \mathbf{v} > 0 \quad \forall \mathbf{v} \neq \mathbf{0}$$

- 由于特征向量 $\mathbf{u}_i$ 构成一个完备集，任意向量 $\mathbf{v}$ 可表示为

$$\mathbf{v} = \sum_i c_i \mathbf{u}_i$$

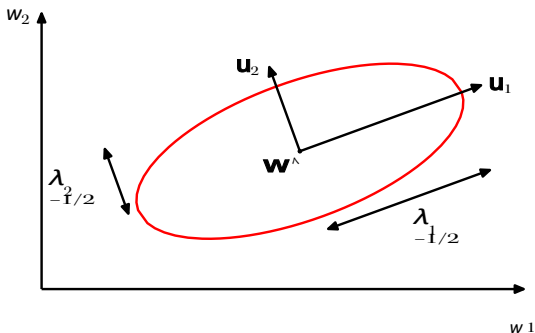
- 我们有

$$\mathbf{v}^T \mathbf{H} \mathbf{v} = \sum_i c_i^2 \lambda_i$$

- 因此，当且仅当H的所有特征值均为正值时，H才是正定矩阵。



误差曲面：局部二次逼近



- 在极小值 w 的邻域内，误差函数可近似为二次函数。
- 恒定误差等值线呈椭圆形，其轴线与海森矩阵的特征向量 $u(i)$ 对齐。
海森矩阵的特征向量 u_i
- 其长度与对应特征值 λ_i 的平方根成反比。



- 求解方程

$$\nabla E_{p,q_0}$$

- 对于神经网络定义的如此复杂的误差函数，
- 大多数迭代数值方法都涉及选择某个初始值 w^{p,q_0} 作为权重向量的初始值
- 然后以如下形式的连续步骤在权重空间中移动：



$$w^{p\tau-1q} \quad w^{p\tau q} \quad \Delta w^{p\tau q}$$



批量梯度下降

- 利用梯度信息的最简方法是选择权重更新为沿负梯度方向的小步长，使得

$$w^{p\tau+1q} = w^{p\tau q} - \eta \nabla E w^{p\tau q}$$

- 其中参数 $\eta > 0$ 称为学习率。
- 每次更新后，需对新权重向量重新计算梯度： $w^{p\tau+1q}$
- 该过程重复进行。



- 误差函数基于训练集定义，
- 因此每一步都需要处理整个训练集才能评估 ∇E 。
- 使用整组数据进行一次处理的技术称为批处理方法。
- 在每个步骤中，权重向量沿着误差函数下降速率最大的方向移动。
- 因此该方法被称为梯度下降法或最陡下降法。



- 深度学习方法能从海量数据集中获益匪浅。
- 然而，当训练集包含大量数据点时，批处理方法可能变得极其低效
- ——因为每次误差函数或梯度计算都需要处理全部数据集。